

La API JDBC

Clases DAO (Data Access Object)

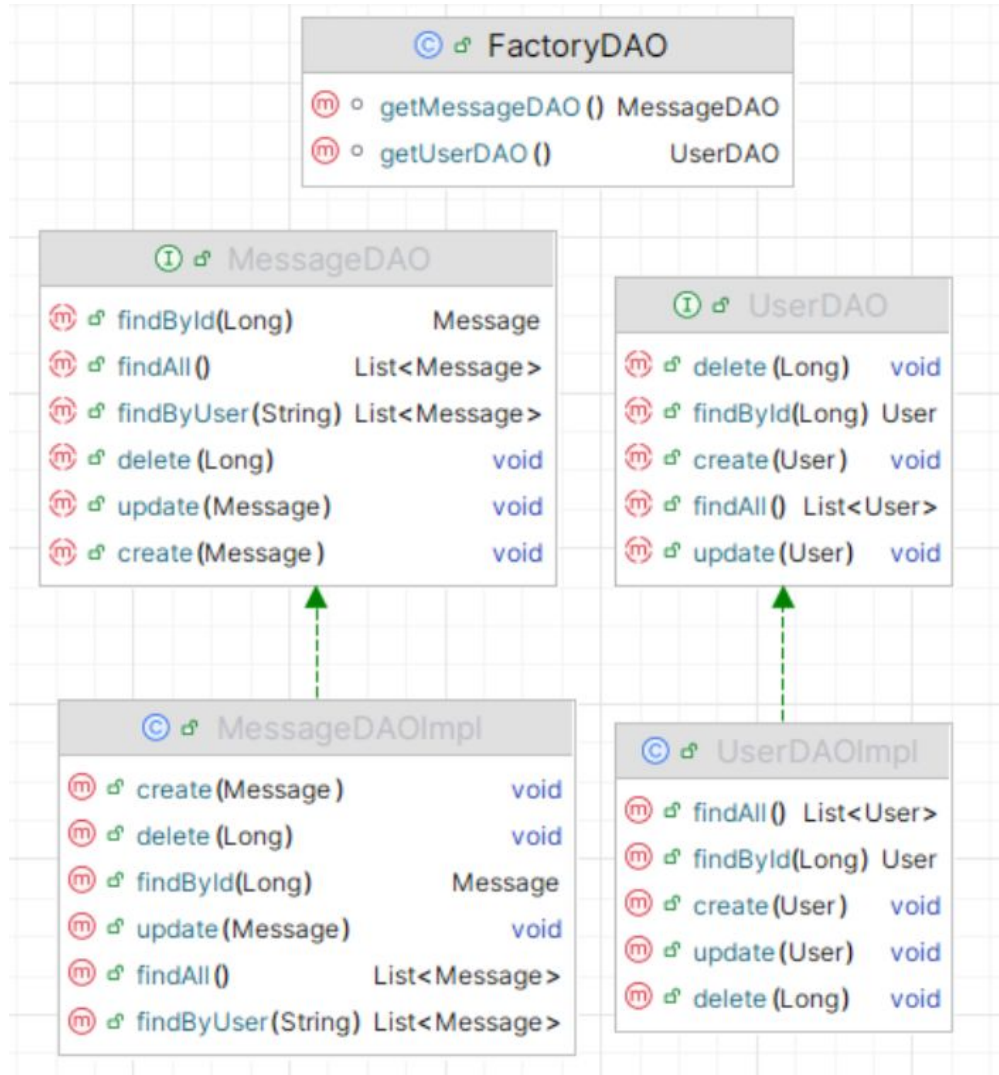
El patrón arquitectónico Data Access Object (DAO), permite separar la capa de lógica de negocios de la capa de acceso a datos, de tal forma que el DAO encapsula toda la lógica de acceso de datos del resto de la aplicación.

Implementar la lógica de acceso a datos en la capa de lógica de negocio puede hacer el código complejo y no extensible. Se recomienda siempre usar DAO para abstraer y encapsular todos los accesos a los datos.

El DAO maneja la conexión con las fuentes de datos para obtener y almacenar datos.

La API JDBC

Clases DAO



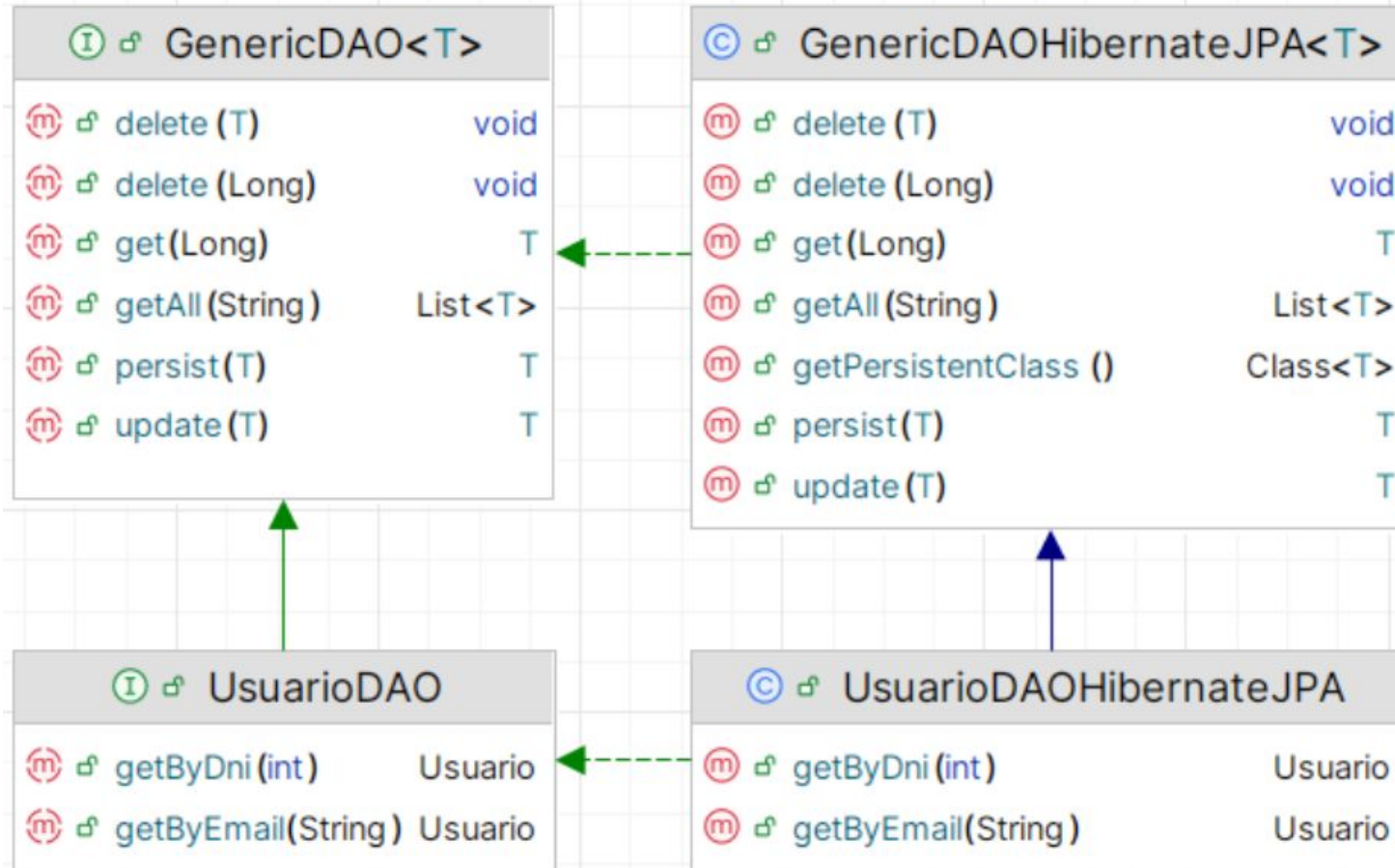
Esta clase crea objetos xxxDAO. Nos provee de objetos que implementan las distintas interfaces xxxDAO

Las interfaces xxxDAO tienen operaciones comunes de acceso a datos.

Implementaciones de las interfaces xxxDAO usando JDBC

DAO Genérico

La interface **GenericDAO** debería incluir los métodos comunes a todas las entidades



Se agregan métodos particulares de la entidad **Usuario**

Se implementan los métodos particulares de **UsuarioDAO**

DAO Genérico

Las interfaces

GenericDAO<T>

delete(Long)	void
delete(T)	void
get(Long)	T
getAll(String)	List<T>
persist(T)	T
update(T)	T



UsuarioDAO<T>

getByDni(int)	T
getByEmail(String)	T

```
package ttps.persistencia.dao;
import java.util.List;

public interface GenericDAO<T> {
    public void delete(T entity);
    public void delete(Long id);
    public T get(Long id);
    public List<T> getAll(String columnOrder);
    public T persist(T entity);
    public T update(T entity);
}
```

```
package ttps.persistencia.dao;
import ttps.modelo.Usuario;

public interface UsuarioDAO<T> extends GenericDAO<T> {
    public T getByDni(int dni);
    public T getByEmail(String email);
}
```

DAO Genérico

```
public class GenericDAOHibernateJPA<T>
    implements GenericDAO<T> {

    protected Class<T> persistentClass;
    public GenericDAOHibernateJPA(Class<T> clase) {
        this.persistentClass = clase;
    }
    public Class<T> getPersistentClass() {
        return persistentClass;
    }

    @Override
    public T persist(T entity) {
        EntityManager em = EMF.getEMF().createEntityManager();
        EntityTransaction tx = null;
        try {
            tx = em.getTransaction();
            tx.begin();
            em.persist(entity);
            tx.commit();
        } catch (RuntimeException e) {
            if (tx != null && tx.isActive()) tx.rollback();
            throw e; // escribir en un log o mostrar mensaje
        } finally {
            em.close();
        }
        return entity;
    }

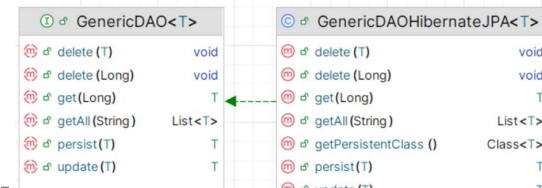
    @Override
    public T update(T entity) {
        EntityManager em = EMF.getEMF().createEntityManager();
        EntityTransaction etx = em.getTransaction();
        etx.begin();
        T entityMerged = em.merge(entity);
        etx.commit();
        em.close();
        return entityMerged;
    }
}
```

```
@Override
public T update(T entity) {
    EntityManager em = EMF.getEMF().createEntityManager();
    EntityTransaction etx = em.getTransaction();
    etx.begin();
    T entityMerged = em.merge(entity);
    etx.commit();
    em.close();
    return entityMerged;
}

@Override
public void delete(T entity) {
    EntityTransaction tx = null;
    try(EntityManager em = EMF.getEMF().createEntityManager()) {
        tx = em.getTransaction();
        tx.begin();
        em.remove(em.merge(entity));
        tx.commit();
    } catch (RuntimeException e) {
        if (tx != null && tx.isActive()) tx.rollback();
        throw e; // escribir en un log o mostrar un mensaje
    }
}

public List<T> getAll(String columnOrder) {
    Query consulta =
        EMF.getEMF().createEntityManager()
            .createQuery("SELECT e FROM "+
                getPersistentClass().getSimpleName() +
                " e order by e." + columnOrder);
    List<T> resultado = (List<T>) consulta.getResultList();
    return resultado;
}

. . .
}
```



DAO Genérico

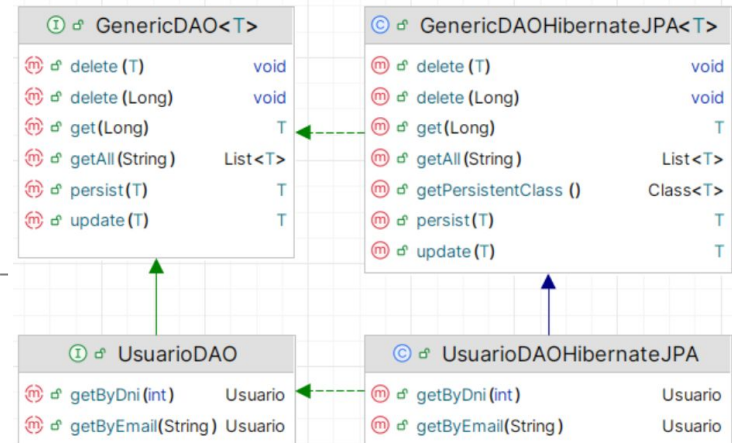
```
ttps.persistencia.dao.impl;
```

```
public class UsuarioDAOHibernateJPA extends GenericDAOHibernateJPA<Usuario>
    implements UsuarioDAO{

    public UsuarioDAOHibernateJPA() {
        super(Usuario.class);
    }

    public Usuario getByDni(int dni){
        . . .
    }

    public Usuario getByEmail(String mail) {
        EntityManager em = EMF.getEMF().createEntityManager();
        Usuario usr;
        try {
            usr = (Usuario) em.createQuery("SELECT m FROM " +
                this.getPersistentClass().getSimpleName() + " m WHERE m.email = :email")
                .setParameter("email", mail).getSingleResult();
        } catch (Exception e) {
            usr = null;
        } finally {
            em.close();
        }
        return usr;
    }
}
```



DAO Genérico

Clases utilitarias

```
package ttps.persistencia;
//imports
public class EMF {
    private static EntityManagerFactory em = null;
    static {
        try {
            em = Persistence.createEntityManagerFactory("unlp");
        } catch (PersistenceException e) {
            System.err.println("Error al crear EntityManagerFactory: "+e.getMessage());
            e.printStackTrace();
        }
    }

    public static EntityManagerFactory getEMF(){
        return em;
    }

    private EMF() {}
}
```

```
package ttps.persistencia;
//imports
public class FactoryDAO {

    public static UsuarioDAO getUsuarioDAO() {
        return new UsuarioDAOHibernateJPA();
    }

    public static UsuarioDAO getUsuarioDAO() {
        return new UsuarioDAOHibernateJPA();
    }
    public static MenuDAO getMenuDAO() {
        return new MenuDAOHibernateJPA();
    }
    . . .
}
```


Hibernate

Java Persistence API (JPA)

JPA introduce la idea de “**unidad de persistencia**”. Una unidad de persistencia provee una manera de especificar un conjunto de clases anotadas o clases que persistirán, junto con las propiedades del proveedor de JPA que se utilizará para esa unidad. La unidad de persistencia tiene un nombre y ese nombre es usado para crear en *run time* un **EntityManagerFactory**.

persistence.xml

```
<persistence xmlns="http://java.sun.com/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
http://java.sun.com/xml/ns/persistence/persistence_2_0.xsd" version="2.0">
<persistence-unit name="unlp">
    <provider>org.hibernate.ejb.HibernatePersistence</provider>
    <class>paquete.clase1</class>
    . . .
    <properties>
        <property name="hibernate.connection.driver_class" value="com.mysql.jdbc.Driver"/>
        <property name="hibernate.connection.password" value="admin"/>
        <property name="hibernate.connection.url" value="jdbc:mysql://localhost:3306/sbarra"/>
        <property name="hibernate.connection.username" value="root" />
        <property name="hibernate.dialect" value="org.hibernate.dialect.MySQL5InnoDBDialect />
        <property name="hibernate.hbm2ddl.auto" value="update"/> #(create,create-drop,update)
    . . .
    </properties>
</persistence-unit>
</persistence>
```

El archivo se ubica en alguna de las carpetas cuyo contenido es enviado a la carpeta **WEB-INF/classes/META-INF**